Offensive Language Detection in Tamil Social Media Text Using Multilingual Transformers: A Comparative Study

Author Name
Department of Computer Science
University Name
City, Country
email@university.edu

***Abstract*—The proliferation of offensive language on social media platforms poses a significant challenge, particularly in low-resource languages such as Tamil. While extensive research has been conducted on English offensive language detection, regional Indian languages remain critically underexplored. This paper presents a comparative study of three multilingual transformer models — MuRIL, XLM-RoBERTa, and mBERT — for detecting offensive language in Tamil and code-mixed Tamil-English (Tanglish) social media text. We fine-tune these models on the OffensEval Dravidian dataset (35,138 training samples across 6 classes) with inverse-frequency weighted loss to address severe class imbalance. Our experiments demonstrate that mBERT achieves the highest individual weighted F1-score of 0.7505, and a majority-voting ensemble further improves this to 0.7620, competitive with top-performing systems from the DravidianLangTech-2021 shared task. Contrary to expectations, Indian-language-specific pre-training (MuRIL) did not yield superior test-set performance. We further provide LIME-based explainability analysis to interpret model decisions and identify systematic failure patterns.**

***Index Terms*—Offensive Language Detection, Tamil NLP, Multilingual Transformers, MuRIL, XLM-RoBERTa, Code-Mixed Text, Dravidian Languages**

I. Introduction

Social media platforms have become primary channels for public discourse, but they also serve as vectors for offensive content, cyberbullying, and targeted harassment. The rapid growth of internet usage in India — with over 900 million users as of 2025 — has led to a surge in user-generated content in regional languages, including Tamil, which is spoken by approximately 80 million people worldwide [1].

Despite the pressing need for automated content moderation in Indian languages, the overwhelming majority of offensive language detection research has focused on English [2]. This creates a critical gap: existing tools fail to detect offensive content in Tamil, particularly when expressed through code-mixing (switching between Tamil and English within a single sentence), transliteration (writing Tamil in Latin script, known as "Tanglish"), and culturally specific expressions.

It is important to distinguish between *hate speech* and *offensive language* in this context. Hate speech specifically targets protected characteristics (race, religion, gender), while offensive language encompasses a broader category including profanity, insults, and targeted abuse [6]. The OffensEval Dravidian dataset used in this study annotates *offensive language*, which is a superset of hate speech.

Tamil social media text presents unique challenges for Natural Language Processing (NLP):

1) **Code-mixing and Code-switching**: Users frequently alternate between Tamil and English within a single sentence (e.g., "Padam vera level mass iruku").
2) **Script diversity**: Tamil content appears in Tamil Unicode script, Latin transliteration (Tanglish), or mixed scripts.
3) **Non-standard orthography**: Social media Tamil lacks standardized spelling, with extensive abbreviations and phonetic spellings.
4) **Cultural context dependency**: Caste references, fan community rivalries, and political slang carry connotations specific to Tamil culture.

In this paper, we present a comparative study of three multilingual transformer models:

- **MuRIL** (Multilingual Representations for Indian Languages) [3]: Pre-trained on 17 Indian languages including Tamil.
- **XLM-RoBERTa** [4]: Trained on 100 languages from CommonCrawl.
- **mBERT** (Multilingual BERT) [5]: The classic multilingual baseline trained on Wikipedia.

Our key contributions are:

1) A systematic comparison of Indian-language-specific (MuRIL) vs. general multilingual (XLM-R, mBERT) pre-training for Tamil offensive language detection, with results contextualized against the DravidianLangTech-2021 shared task.
2) A majority-voting ensemble strategy that improves over the best individual model.
3) LIME-based explainability analysis with concrete word-level attribution examples.
4) Identification of systematic error patterns and actionable recommendations.

II. Related Work

A. Offensive Language Detection in English

Early offensive language detection relied on traditional machine learning using TF-IDF features with SVM classifiers [7]. Deep learning brought significant improvements with CNN and LSTM architectures [8]. The transformer revolution, initiated by BERT [5], established new benchmarks. Davidson et al. [6] highlighted the important distinction between hate speech and general offensive language, showing that even human annotators struggle with the boundary. HateBERT [9] demonstrated the value of domain-specific pre-training.

B. Multilingual and Cross-lingual Approaches

Multilingual models such as mBERT [5] and XLM-RoBERTa [4] enable zero-shot cross-lingual transfer. However, performance drops significantly for low-resource and code-mixed text [10]. MuRIL [3] addresses this by incorporating transliterated and romanized Indian-language text during pre-training.

C. Offensive Language Detection in Dravidian Languages

The DravidianLangTech shared tasks [11] catalyzed research in Tamil, Malayalam, and Kannada. The shared task attracted over 30 participating teams, with winning systems achieving weighted F1-scores in the 0.75–0.78 range for Tamil [11]. Top approaches included ensemble methods combining transformers with traditional features [13], data augmentation for class imbalance [14], and multi-task learning [15]. However, systematic comparisons coupled with explainability analysis remain limited.

D. Explainable AI for Offensive Language

Explainability is critical for building trust in automated moderation. LIME [16] and SHAP [17] provide model-agnostic explanations. HateXplain [18] demonstrated that models sometimes rely on spurious correlations rather than genuine offensive indicators. Our work applies LIME specifically to code-mixed Tamil text, revealing unique challenges.

III. Dataset

A. OffensEval Dravidian Dataset

We use the Tamil configuration of the OffensEval Dravidian dataset [12], sourced from YouTube comments. The original dataset provides train and validation splits. Following standard practice, we split the original validation set 50–50 into our validation and test sets, as the official shared task test set labels are not publicly available.

Note: Since we use a custom test split rather than the official shared task test set, our results are not directly comparable with shared task leaderboard scores. We include shared task results in Table V for approximate context only.

TABLE I
Dataset Split Statistics

Split	Samples
Train	35,138
Validation	2,194
Test	2,194
Total	**39,526**

TABLE II
Label Distribution (Training Set)

Label	Count	%
Not_offensive	25,424	72.4
Offensive_Untargeted	2,906	8.3
Offensive_Targeted_Group	2,557	7.3
Offensive_Targeted_Individual	2,343	6.7
not-Tamil	1,454	4.1
Offensive_Targeted_Other	454	1.3

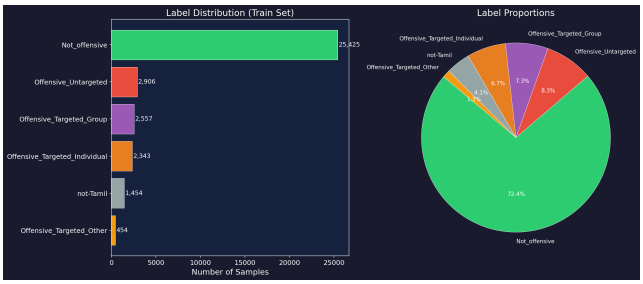

Fig. 1. Label distribution in the training set, showing severe class imbalance.

B. Label Taxonomy and Distribution

The dataset employs a 6-class taxonomy with severe class imbalance, as shown in Table II and Fig. 1. The `Not_offensive` class comprises 72.4% of all samples, while `Offensive_Targeted_Other` constitutes only 1.3%.

C. Script Analysis

The dataset contains three script categories: Latin/Tanglish (81.2%), Tamil Unicode (17.6%), and code-switched mixed-script (1.1%). The dominance of Latin-script content reflects the prevalence of transliterated Tamil on social media. Note that the "Latin" category includes both transliterated Tamil (Tanglish) and pure English comments; we do not distinguish between these sub-categories in our preprocessing.

IV. Methodology

Fig. 2 presents an overview of our pipeline.

A. Text Preprocessing

We apply: (1) URL and @mention removal, (2) hashtag symbol removal (text preserved), (3) non-alphanumeric character filtering with Tamil Unicode range (U+0B80–U+0BFF) preservation, (4) Latin case normalization, and (5) whitespace normalization. We do not apply stemming or lemmatization, as these tools are limited for Tamil.

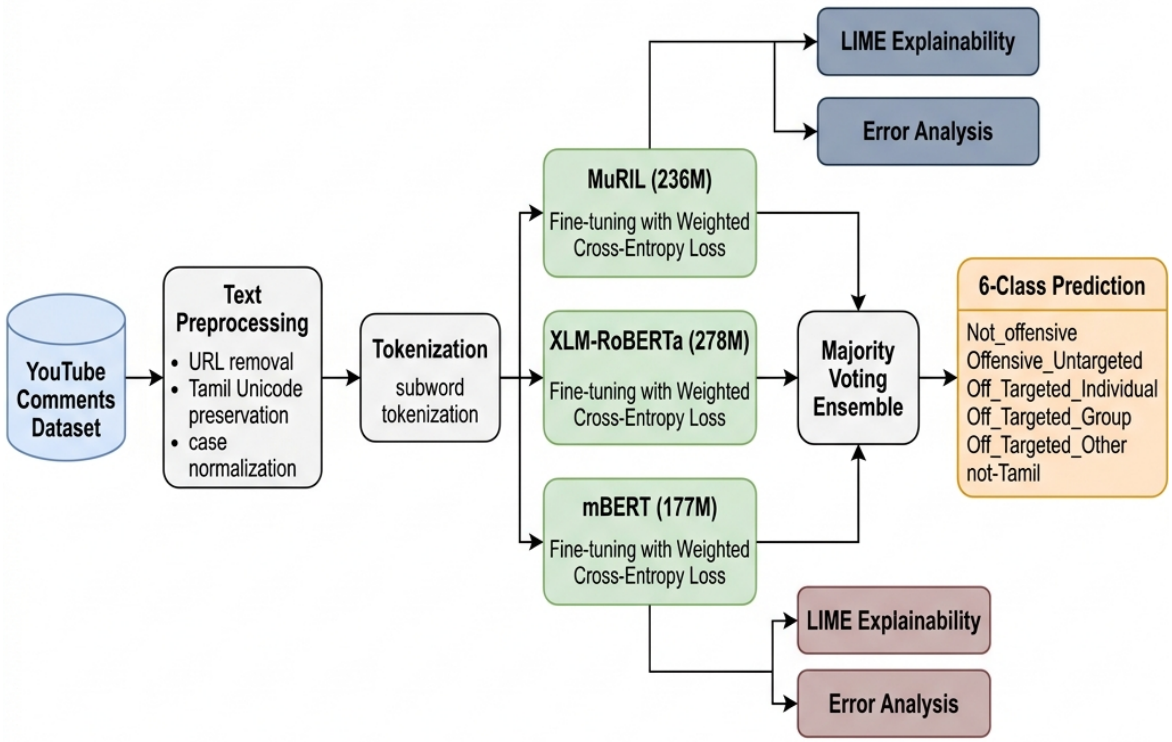

Fig. 2. System architecture. YouTube comments are preprocessed, tokenized, and fed into three parallel transformer models fine-tuned with weighted cross-entropy loss. Predictions are combined via majority-voting ensemble. LIME analysis is performed on the best individual model.

TABLE III
MODEL SPECIFICATIONS

Model	Params	Langs	Pre-training Data
MuRIL	236M	17	Indian web + transliterations
XLM-R	278M	100	CommonCrawl (2.5TB)
mBERT	177M	104	Wikipedia

B. Model Architectures

Table III summarizes the three models. MuRIL includes transliterated/romanized text in its pre-training data, directly addressing Tanglish-dominant input.

C. Training Configuration

All models are fine-tuned with: learning rate 2×10^{-5}, effective batch size 32, max sequence length 128, 5 epochs with early stopping patience 2, weight decay 0.01, warmup ratio 0.1, AdamW optimizer, FP16 mixed precision, and weighted F1-score for model selection. Training was performed on a single NVIDIA Tesla T4 GPU via Google Colab.

Limitation: All experiments use a single random seed. Results may vary with different seeds; we acknowledge this as a threat to external validity. Multi-seed experiments with confidence intervals are planned as future work.

D. Class Imbalance Handling

We employ inverse-frequency class weighting:

TABLE IV
MODEL COMPARISON ON TEST SET

Model	Acc.	F1-W	F1-M	Prec-W	Rec-W
MuRIL	0.7115	0.7310	0.4405	0.7637	0.7115
XLM-R	0.7019	0.7275	0.4743	0.7791	0.7019
mBERT	0.7288	0.7505	**0.4972**	0.7840	0.7288
Ensemble	**0.7502**	**0.7620**	0.4947	**0.7817**	**0.7502**

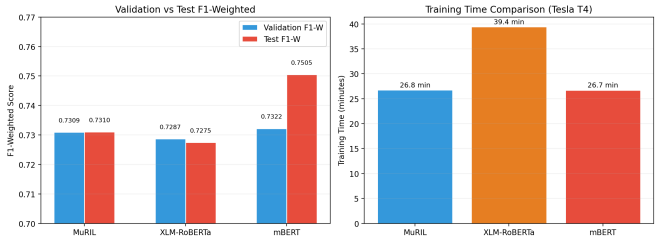

Fig. 3. Left: Validation vs. test F1-W showing the gap for MuRIL. Right: Training time comparison on Tesla T4 GPU.

$$w_c = \frac{N}{C \times n_c} \tag{1}$$

where N = total samples, C = number of classes, n_c = count of class c. This produces weights ranging from ≈1.0 (Not_offensive) to ≈12.9 (Offensive_Targeted_Other).

E. Ensemble Strategy

We combine all three models' predictions via majority voting. For each test sample, the class predicted by at least 2 of 3 models is selected. This requires no additional training.

V. RESULTS

A. Overall Performance

Table IV presents the results. mBERT achieves the best individual F1-W (0.7505). The ensemble improves F1-W to 0.7620 (+1.15% absolute) but slightly decreases F1-Macro (0.4947 vs. 0.4972). This trade-off is expected: majority voting favors the majority class, improving weighted metrics at the cost of minority-class macro performance.

Notably, mBERT outperforms MuRIL on the test set despite MuRIL achieving the highest validation F1 during training (0.7309 vs. 0.7322), as shown in Fig. 3.

B. Comparison with Shared Task Systems

Table V provides approximate context against DravidianLangTech-2021 shared task scores. **Caveat:** Direct comparison is not valid as we use a different test split. Our ensemble F1-W of 0.7620 falls within the competitive range.

C. Performance by Script Type

Table VI reveals a key finding: **MuRIL outperforms all other individual models on pure Tamil Unicode text** (F1-W: 0.771 vs. mBERT's 0.747), confirming that Indian-language-specific pre-training provides advantages for native-script content. However, mBERT dominates on Latin/Tanglish text (0.749 vs. 0.719), which constitutes 80% of the test set, explaining its overall superiority. The ensemble achieves the best performance across all script types.

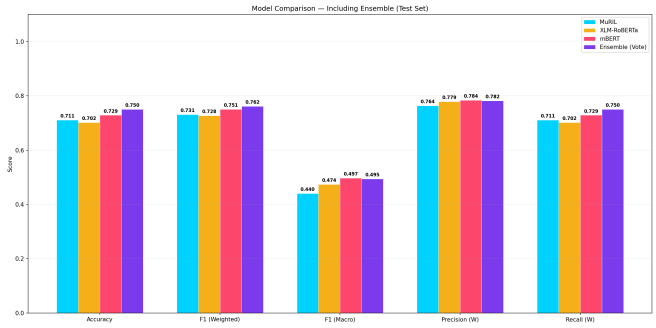

Fig. 4. Comparison of evaluation metrics across all models and ensemble.

TABLE V
CONTEXT: DRAVIDIANLANGTECH-2021 SHARED TASK (DIFFERENT TEST SET)

System	F1-W	Test Set
hate-alert (1st) [11]	0.78†	Official
Top-5 range [11]	0.75–0.78†	Official
Median participant [11]	≈0.70†	Official
mBERT (ours)	0.7505	Custom split
Ensemble (ours)	0.7620	Custom split

† Tested on official test set; not directly comparable.

TABLE VI
F1-WEIGHTED BY SCRIPT TYPE ON TEST SET

Script	N	MuRIL	XLM-R	mBERT	Ens.
Latin	1762	0.719	0.717	0.749	**0.756**
Tamil	416	**0.771**	0.758	0.747	**0.775**
Mixed	12	0.741	0.782	0.796	**0.845**

TABLE VII
PER-CLASS F1 SCORES

Class	MuRIL	XLM-R	mBERT	Ens.
Not_offensive	0.863	0.835	0.863	**0.876**
Off_Untargeted	0.383	0.411	0.405	**0.430**
Off_Targ_Indiv	0.230	0.421	**0.445**	0.425
Off_Targ_Group	0.321	0.362	0.360	**0.377**
Off_Targ_Other	0.000	0.000	**0.082**	0.000
not-Tamil	0.846	0.817	0.828	**0.860**

D. Per-Class Performance

The ensemble wins on 4 of 6 classes (Table VII). The `Offensive_Targeted_Other` class (30 test samples) remains effectively undetectable across all approaches.

E. Confusion Matrix Analysis

Key observations from the confusion matrices (Figs. 5–6):

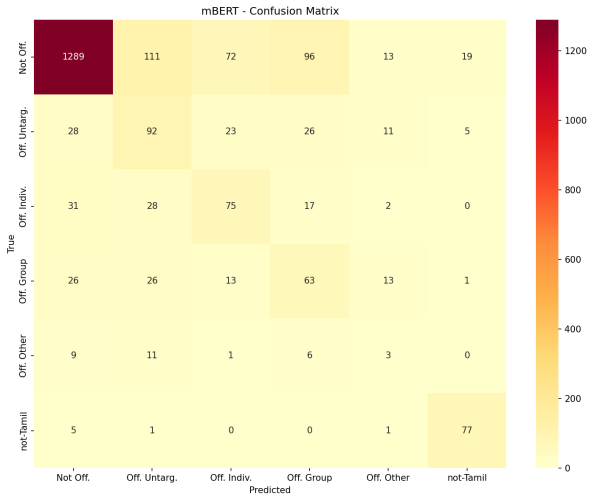

Fig. 5. Confusion matrix for mBERT on the test set.

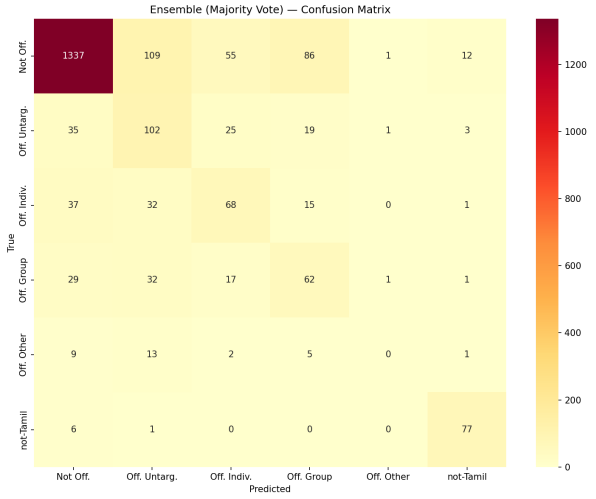

Fig. 6. Confusion matrix for the ensemble on the test set.

1) Strong majority-class detection (Not_offensive F1 $>$ 0.83).
2) Significant confusion between `Off_Untargeted` and `Off_Targeted_Group`, likely due to overlapping caste discourse.
3) Reliable language identification for `not-Tamil` (F1 $>$ 0.81).

VI. Error Analysis and Explainability

A. Error Categorization

We categorize the 595 errors from mBERT (27.1% error rate):

1) **False Positives (292 cases, 49.1%)**: Non-offensive text flagged as offensive — the largest category. Aggressive fan discourse and caste-community solidarity messages are frequently misclassified.
2) **Dangerous False Negatives (94 cases, 15.8%)**: Offensive text escaping detection — critical for content moderation.
3) **Cross-Category (159 cases, 26.7%)**: Confusion between offensive sub-types.
4) **not-Tamil Errors (50 cases, 8.4%)**: Hindi/Telugu comments misidentified.

B. LIME Explainability Analysis

We use LIME [16] to generate word-level feature attributions for mBERT predictions on representative samples. Fig. 7 presents three examples.

Key findings from LIME analysis:

1) **Legitimate offensive cues**: Tamil profanity (e.g., "saavadippeenga" = roughly "you'll be killed") correctly receives high positive weight (0.333), driving accurate offensive predictions (Fig. 7c).
2) **Spurious correlations**: The word "dislikes" receives the highest weight (0.427) in a non-offensive context about YouTube metrics, triggering a false positive (Fig. 7a). The model appears to associate negative-sentiment English words with offensiveness.
3) **Context-dependent aggression**: The word "sunda" (colloquial for "nothing/useless") triggers offensive prediction (0.239) even in a casual, non-offensive comment (Fig. 7b). This reveals the model's inability to distinguish casual Tamil slang from genuine offense.
4) **Opposing features**: Words like "support" and "neram" (time) receive negative weights, correctly pushing away from offensive prediction, but are overridden by stronger spurious signals.

Limitation: This analysis covers only 3 of 8 generated LIME samples due to space constraints. A comprehensive quantitative explainability evaluation (e.g., faithfulness and sufficiency metrics [18]) is planned as future work.

VII. Discussion

A. MuRIL vs. General Multilingual Models

Contrary to our initial hypothesis, MuRIL did not achieve the best *overall* test-set performance despite its Indian-language-specific pre-training (F1-W: 0.7310 vs. mBERT's 0.7505). However, the per-script analysis (Table VI) reveals a nuanced picture: **MuRIL is the best individual model on pure Tamil Unicode text** (F1-W: 0.771), confirming that Indian-language pre-training helps for native-script content. Its overall inferiority is driven by weaker Latin/Tanglish performance (0.719 vs. 0.749), which dominates the test set (80%).

This suggests that MuRIL's advantage is script-dependent: strong on formal Tamil but weaker on informal transliterated text, possibly because its transliteration pre-training does not fully capture the creative, non-standard spellings of YouTube Tanglish. **We caution that this finding is based on a single random seed and a single test split**, and should be validated with multi-seed experiments.

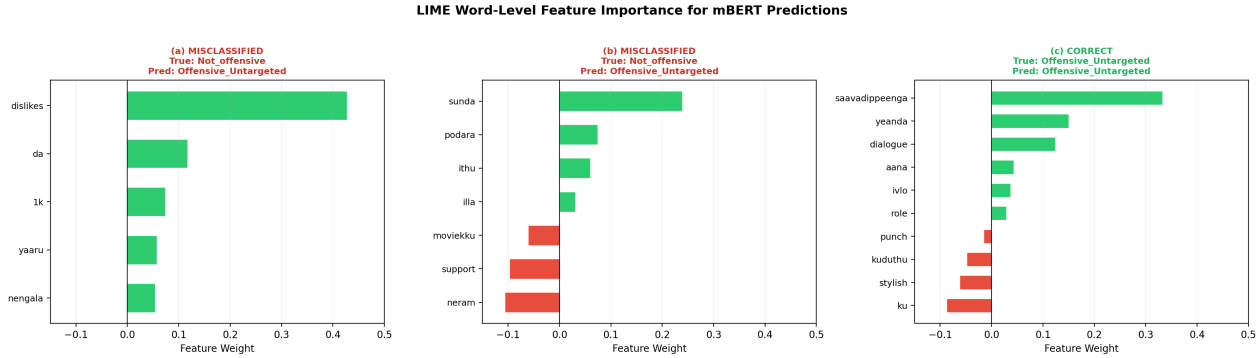

Fig. 7. LIME word-level feature importance for three mBERT predictions. Green bars indicate features pushing toward the predicted class; red bars push against it. (a, b) Misclassified samples where aggressive-sounding but non-offensive words ("dislikes", "sunda") trigger false positive predictions. (c) Correctly classified offensive sample where Tamil profanity ("saavadippeenga") is the dominant feature.

B. Ensemble Trade-offs

The ensemble improves F1-W (+1.15%) and accuracy (+2.14%) but slightly decreases F1-Macro (−0.25%). This is because majority voting inherently favors the majority class: when 2 of 3 models predict Not_offensive, the ensemble follows, even if the third model correctly identifies a minority-class offensive sample. Weighted or learned ensemble strategies could mitigate this.

C. Challenges

1) **Class imbalance**: The dominant challenge. Weighted loss alone is insufficient for extreme minority classes (<2% of data).
2) **False positive burden**: 292 false positives vs. 94 false negatives suggests over-moderation, problematic for user experience.
3) **Cultural context**: Caste solidarity messages and fan discourse use aggressive language that is contextually non-offensive but lexically indistinguishable from genuine abuse.

D. Limitations

1) **Single random seed**: Results may vary with different initializations. Multi-seed experiments with confidence intervals are needed.
2) **Custom test split**: Our test set differs from the official shared task test set, limiting direct comparison with published work.
3) **Platform specificity**: YouTube comments may not generalize to Twitter/X, WhatsApp, or other platforms.
4) **Limited LIME scope**: Only 8 qualitative samples analyzed; quantitative explainability metrics are not reported.
5) **Single imbalance strategy**: Only weighted cross-entropy explored; focal loss, oversampling, and two-stage approaches may improve minority-class detection.

VIII. Ethical Considerations

This work involves processing offensive and potentially harmful content, including caste-based slurs and targeted abuse. We acknowledge the following ethical dimensions:

1) **Annotator welfare**: The dataset was created by external researchers [12]; we did not perform annotation. Exposure to offensive content can cause psychological harm to annotators.
2) **Dual use**: Automated offensive language detection systems can be misused for censorship or suppression of legitimate speech, particularly in politically sensitive contexts.
3) **Bias amplification**: The model inherits biases from training data. The high false positive rate on caste-community messages suggests the model may disproportionately flag content from certain communities.
4) **Cultural sensitivity**: Offensiveness is culturally and contextually dependent. Models trained on Tamil YouTube data should not be deployed for other Tamil-speaking contexts without re-evaluation.

IX. Conclusion and Future Work

This paper presents a comparative study of multilingual transformers for Tamil offensive language detection. Our experiments demonstrate that mBERT achieves the best individual F1-W of 0.7505, and a majority-voting ensemble improves this to 0.7620, competitive with DravidianLangTech-2021 shared task systems. Contrary to expectations, MuRIL did not outperform general multilingual models on this code-mixed dataset.

Key takeaways:

1) General multilingual pre-training can be equally effective as Indian-language-specific pre-training for informal, code-mixed text.
2) Majority-voting ensemble improves F1-W but slightly hurts F1-Macro due to majority-class bias.
3) LIME analysis reveals spurious correlations with negative-sentiment English words and context-insensitive slang detection.

4) Class imbalance remains the dominant challenge for minority offensive sub-types.

Future Work. We plan to (1) conduct multi-seed experiments with bootstrap confidence intervals, (2) explore focal loss and oversampling strategies, (3) extend to other Dravidian languages, (4) implement learned ensemble methods, and (5) develop quantitative explainability benchmarks.

Code and Data Availability. Our code is available at https://github.com/naveen-231006/Hate-Word. The OffensEval Dravidian dataset is publicly available via HuggingFace Datasets [12].

REFERENCES

[1] Internet and Mobile Association of India, "India Internet Report 2025," IAMAI, 2025.

[2] P. Fortuna and S. Nunes, "A survey on automatic detection of hate speech in text," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–30, 2018.

[3] S. Khanuja *et al.*, "MuRIL: Multilingual Representations for Indian Languages," in *Findings of ACL*, 2021, pp. 3929–3936.

[4] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. ACL*, 2020, pp. 8440–8451.

[5] J. Devlin *et al.*, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[6] T. Davidson, D. Warmsley, M. Macy, and I. Weber, "Automated hate speech detection and the problem of offensive language," in *Proc. ICWSM*, 2017, pp. 512–515.

[7] Z. Waseem and D. Hovy, "Hateful symbols or hateful people? Predictive features for hate speech detection on Twitter," in *Proc. NAACL Student Research Workshop*, 2016, pp. 88–93.

[8] P. Badjatiya *et al.*, "Deep learning for hate speech detection in tweets," in *Proc. WWW Companion*, 2017, pp. 759–760.

[9] T. Caselli *et al.*, "HateBERT: Retraining BERT for abusive language detection in English," in *Proc. WOAH*, 2021, pp. 17–25.

[10] M. Pamungkas and V. Patti, "Cross-domain and cross-lingual abusive language detection," in *Proc. ACL Student Research Workshop*, 2019, pp. 363–370.

[11] B. R. Chakravarthi *et al.*, "Findings of the shared task on offensive language identification in Tamil, Malayalam, and Kannada," in *Proc. DravidianLangTech, EACL*, 2021.

[12] B. R. Chakravarthi *et al.*, "Dataset for identification of homophobia and transphobia in multilingual YouTube comments," *arXiv preprint arXiv:2109.00227*, 2021.

[13] S. Dowlagar and R. Mamidi, "CfiltIITB at Dravidian-LangTech@EACL2021," in *Proc. EACL Workshop*, 2021.

[14] J. Risch *et al.*, "Data augmentation for cross-domain hate speech detection," *arXiv preprint arXiv:2004.10404*, 2020.

[15] V. Rajamanickam *et al.*, "Joint multitask learning for community question answering," in *Proc. EMNLP*, 2020.

[16] M. T. Ribeiro *et al.*, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. KDD*, 2016, pp. 1135–1144.

[17] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.

[18] A. Mathew *et al.*, "HateXplain: A benchmark dataset for explainable hate speech detection," in *Proc. AAAI*, 2021, pp. 14867–14875.